



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №8

з дисципліни «Технології розроблення програмного забезпечення»

Тема: «13. Office communicator»

Виконав:

студент групи ІА-32
Шадлер Андрій

Перевірив:

вик. Мягкий М. Ю.

Тема роботи

Використання шаблону проєктування Composite (Компоновщик) у програмному забезпеченні.

Мета роботи

Ознайомитися з шаблоном проєктування Composite, його призначенням та структурою, а також реалізувати даний шаблон у програмному проєкті з метою уніфікованої роботи з одиничними об'єктами та їх композиціями.

Зміст

Теоретичні відомості.....	2
Хід виконання:.....	2
Завдання:	2
Діаграма класів:	3
Опис реалізації:	3
Вихідний код	7
Контрольні запитання.....	12
Висновок	14

Теоретичні відомості

Шаблон проєктування Composite

Composite (Компоновщик) – це структурний шаблон проєктування, який дозволяє клієнтському коду однаково працювати як з окремими об'єктами, так і з їх ієрархіями (деревоподібними структурами).

Основна ідея шаблону полягає у створенні спільного інтерфейсу для:

- одиничних об'єктів (Leaf),
- складених об'єктів (Composite),

що дозволяє клієнту не розрізняти ці типи під час використання.

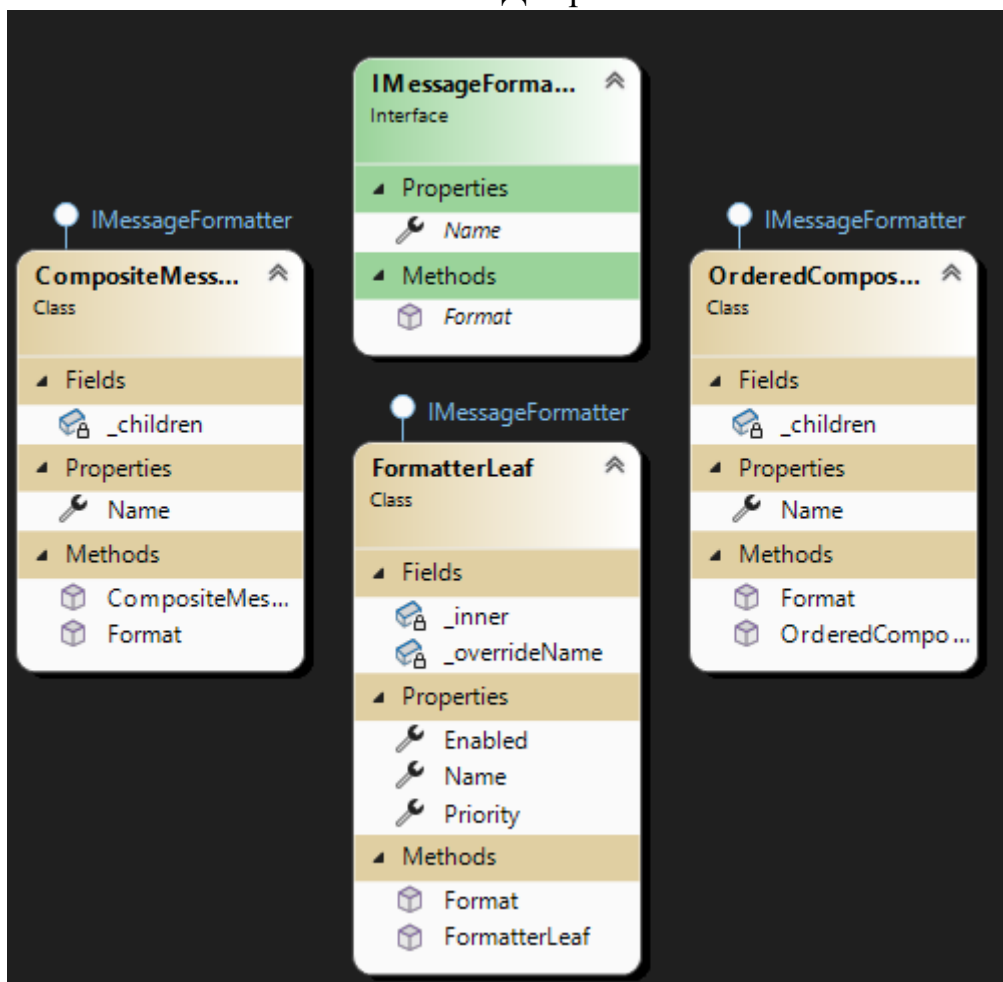
Хід виконання:

Завдання:

- Ознайомитись з короткими теоретичними відомостями.

- Реалізувати частину функціоналу робочої програми у вигляді класів та їхньої взаємодії для досягнення конкретних функціональних можливостей.
- Реалізувати один з розглянутих шаблонів за обраною темою.
- Реалізувати не менше 3-х класів відповідно до обраної теми.
- Підготувати звіт щодо виконання лабораторної роботи. Поданий звіт повинен містити: діаграму класів, яка представляє використання шаблону в реалізації системи, навести фрагменти коду по реалізації цього шаблону.

Діаграма класів:



Опис реалізації:

Опис реалізації в лабораторній роботі

У даній лабораторній роботі шаблон Composite було реалізовано для задачі форматування повідомлень у чаті.

Метою реалізації є можливість трактувати групу форматерів повідомлень як один форматер, не змінюючи клієнтський код.

Компонент (Component)

В ролі компонента виступає інтерфейс:

`IMessageFormatter`

Він визначає спільний контракт форматування повідомлення та використовується як для одиничних форматерів, так і для композиції.

Листові елементи (Leaf)

Листами композиції є окремі реалізації форматування, зокрема:

- форматери простого тексту;
- форматери Markdown;
- форматери, реалізовані через Bridge або Adapter.

Ці класи реалізують інтерфейс `IMessageFormatter` та не містять дочірніх елементів.

Композит (Composite)

Було створено клас:

`CompositeMessageFormatter`

який:

- реалізує інтерфейс `IMessageFormatter`;
- містить колекцію дочірніх форматерів (`IMessageFormatter`);
- послідовно викликає метод `Format()` для кожного дочірнього елемента;
- об'єднує результати форматування в один фінальний рядок.

Таким чином, `CompositeMessageFormatter` виступає роллю `Composite`.

Клієнтський код

Клієнтський код (RepositoryMessages) працює з інтерфейсом IMessageFormatter і не має інформації про те, чи використовується одиничний формater, чи композиція з декількох формaterів.

Це забезпечує прозорість використання шаблону Composite.

Відповідність шаблону Composite

Реалізація відповідає шаблону Composite з таких причин:

1. Єдиний інтерфейс для одиничних об'єктів і композицій.
2. Composite містить Component того ж типу.
3. Клієнт не розрізняє Leaf і Composite.
4. Деревоподібна структура може бути розширена без зміни клієнтського коду.

Переваги реалізації

- уніфікована робота з одиничними та складеними формaterами;
- зменшення складності клієнтського коду;
- можливість гнучкого комбінування формaterів;
- покращена розширюваність системи.

Обмеження та зауваження

- порядок застосування дочірніх формaterів може впливати на результат;
- склад композиції у демонстраційній реалізації визначається автоматично через контейнер залежностей;
- у реальних системах доцільно явно конфігурувати склад та порядок дочірніх компонентів.

Можливі напрямки розширення

Реалізація шаблону Composite може бути вдосконалена шляхом:

- конфігурації списку дочірніх формaterів через налаштування;

- додавання умовних вузлів композиції;
- побудови ієрархічних композицій форматерів;
- поєднання Composite з іншими шаблонами (Strategy, Bridge).

Вихідний код

Formatting/CompositeMessageFormatter.cs

```
using System.Linq;
```

```
using System.Text;
```

```
using Skype.Data;
```

```
namespace Skype.Formatting
```

```
{
```

```
    // Composite: implements IMessageFormatter and delegates to many  
    IMessageFormatter children.
```

```
    public class CompositeMessageFormatter : IMessageFormatter
```

```
    {
```

```
        public string Name => "composite-all";
```

```
        private readonly IEnumerable<IMessageFormatter> _children;
```

```
        public CompositeMessageFormatter(IEnumerable<IMessageFormatter>  
children)
```

```
        {
```

```
            // exclude self by name if DI registered together
```

```
            _children = (children ?? Enumerable.Empty<IMessageFormatter>())
```

```
                .Where(c => c != null && c.Name != Name)
```

```
                .ToList();
```

```
        }
```

```
        public string Format(Message message)
```

```
        {
```

```
            var sb = new StringBuilder();
```

```
            var first = true;
```

```

        foreach (var child in _children)
        {
            if (!first) sb.AppendLine().AppendLine("---"); // visual separator
            sb.Append(child.Format(message));
            first = false;
        }

        return sb.ToString();
    }
}

```

Formatting/OrderedCompositeFormatter.cs

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using Skype.Data;

namespace Skype.Formatting
{
    // Composite that orders children by FormatterLeaf.Priority (defaults to 0 for
    non-leaf children).

    // It concatenates outputs and skips disabled leaves or empty outputs.
    public class OrderedCompositeFormatter : IMessageFormatter
    {
        public string Name => "composite-ordered";
    }
}

```



```

private readonly IReadOnlyList<IMessageFormatter> _children;

public OrderedCompositeFormatter(IEnumerable<IMessageFormatter>
children)
{
    if (children == null) throw new
ArgumentNullException(nameof(children));

    // Filter out possible self-registration and nulls, then order by priority
(ascending).
    _children = children
        .Where(c => c != null && c.Name != Name)
        .OrderBy(c => c is FormatterLeaf fl ? fl.Priority : 0)
        .ToList();
}

public string Format(Message message)
{
    var sb = new StringBuilder();
    var first = true;

    foreach (var child in _children)
    {
        // If child is a leaf and disabled, skip it.
        if (child is FormatterLeaf leaf && !leaf.Enabled) continue;

        var part = child.Format(message);
        if (string.IsNullOrEmpty(part)) continue;

        if (!first)

```

```

        {
            sb.AppendLine().AppendLine("---");
        }

        sb.Append(part);
        first = false;
    }

    return sb.ToString();
}
}
}

```

Formatting/FormatterLeaf.cs

```
using System;
```

```
namespace Skype.Formatting
```

```

{
    // Leaf wrapper: provides ordering and enabled flag for existing
    IMessageFormatter instances.

    public class FormatterLeaf : IMessageFormatter
    {
        private readonly IMessageFormatter _inner;
        private readonly string? _overrideName;

        public int Priority { get; }
        public bool Enabled { get; set; }
    }
}

```

```

    public FormatterLeaf(IMessageFormatter inner, int priority = 0, bool enabled
= true, string? name = null)
    {
        _inner = inner ?? throw new ArgumentNullException(nameof(inner));
        Priority = priority;
        Enabled = enabled;
        _overrideName = name;
    }

    // Expose child's name by default, or an override if provided.
    public string Name => _overrideName ?? _inner.Name;

    public string Format(Message message)
    {
        if (!Enabled)
        {
            return string.Empty;
        }

        return _inner.Format(message);
    }
}

```

Контрольні запитання

1. Яке призначення шаблону «Композит»?

Шаблон Composite (Композит) призначений для уніфікованої роботи з окремими об'єктами та їх композиціями, дозволяючи клієнтському коду не розрізняти одиничні об'єкти і групи об'єктів.

2. Структура шаблону «Композит»

Client

|

v

Component (interface)

^

|

+-----+

| |

Leaf Composite

|

v

Component*

3. Класи шаблону «Композит» та їх взаємодія

- Component – спільний інтерфейс для всіх елементів.
- Leaf – одиничний об'єкт без дочірніх елементів.
- Composite – містить колекцію Component та реалізує їх обробку.
- Client – працює з Component, не розрізняючи Leaf і Composite.

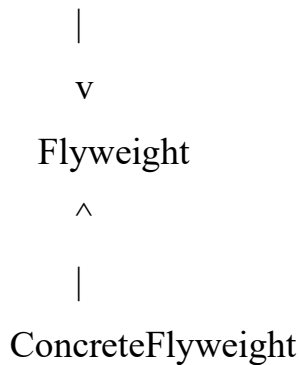
Composite делегує виклики дочірнім елементам.

4. Яке призначення шаблону «Легковаговик»?

Шаблон Flyweight (Легковаговик) призначений для зменшення споживання пам'яті шляхом спільного використання об'єктів, що мають спільний внутрішній стан.

5. Структура шаблону «Легковаговик»

Client ---> FlyweightFactory



6. Класи шаблону «Легковаговик» та їх взаємодія

- Flyweight – інтерфейс легковагового об'єкта.
- ConcreteFlyweight – містить спільний (intrinsic) стан.
- FlyweightFactory – створює та керує пулом Flyweight.
- Client – передає зовнішній (extrinsic) стан під час викликів.

7. Яке призначення шаблону «Інтерпретатор»?

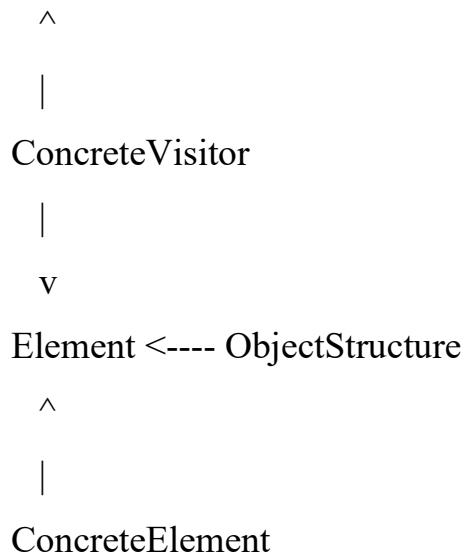
Шаблон Interpreter (Інтерпретатор) призначений для визначення граматики простої мови та реалізації інтерпретатора, який здатен обробляти та виконувати вирази цієї мови.

8. Яке призначення шаблону «Відвідувач»?

Шаблон Visitor (Відвідувач) дозволяє додавати нові операції до структури об'єктів без зміни класів цих об'єктів.

9. Структура шаблону «Відвідувач»

Visitor



10. Класи шаблону «Відвідувач» та їх взаємодія

- Visitor – інтерфейс відвідувача.
- ConcreteVisitor – реалізує операції для кожного типу елементів.
- Element – оголошує метод Accept(Visitor).
- ConcreteElement – реалізує Accept.
- ObjectStructure – колекція елементів.

Visitor виконує операції над елементами через подвійний диспетчинг.

Висновок

У ході виконання лабораторної роботи було реалізовано шаблон проєктування **Composite**, який дозволив однаково працювати з окремими форматерами повідомлень та їх композиціями. Застосування шаблону забезпечило гнучкість архітектури, зменшення зв'язаності клієнтського коду та підвищення розширюваності програмного забезпечення.